

Informe de evidencias de API 3

Integraciones Web

Proyecto TurnosRed

Estudiante Darío

21 de septiembre de 2026

Repositorio de entrega: github.com/drv666/TurnosRed/tree/api3

Resultado de la verificación

La colección de API 3 ejecutada en Postman completó 47 solicitudes y 117 pruebas aprobadas, con 0 fallidas y 0 errores. Las evidencias muestran los endpoints de bienvenida, especialidades y profesionales, junto con las respuestas de validación y de rutas inexistentes.

Alcance de la implementación

La API organiza las rutas de Especialidades y Profesionales en controladores independientes. Un controlador general atiende la bienvenida y las rutas no contempladas. Los servicios administran arreglos inicializados desde archivos JSON en `src/data`. Las altas, modificaciones y bajas de estas dos entidades son ficticias y duran hasta que se reinicia el servidor.

Los métodos de estos controladores son `async`, usan una variable `status` y bloques `try-catch`. Las validaciones preceden a las operaciones; los errores se expresan como JSON con `status`, `message`, `code` y `details`. Cada respuesta tiene `return` explícito. Las bajas exitosas usan `end` porque una respuesta HTTP 204 no debe incluir JSON.

Organización del código

- `src/routes`: definición de los endpoints y conexión con cada controlador.
- `src/controllers`: validaciones de entrada y respuestas HTTP.
- `src/services`: consultas y cambios en los arreglos; unicidad de nombres y matrículas y existencia de la especialidad asociada.
- `src/models` y `src/data`: tipos y datos iniciales de Especialidades y Profesionales.
- `postman/turnos-red-api3-completa.postman_collection.json`: colección importable con sus comprobaciones.

Reproducción de las pruebas

Desde la carpeta del proyecto, instalar las dependencias con `npm.cmd ci`. Copiar `.env.example` a `.env` y ejecutar `npm.cmd run dev`. La configuración de API 3 utiliza el puerto 3003. En PowerShell también puede iniciarse con los siguientes comandos:

```
$env:PORT="3003"
node --watch --import tsx src/server.ts
```

Importar la colección de API 3 en Postman y ejecutarla completa en Runner, en orden y con una iteración. La variable `api3BaseUrl` apunta a `http://localhost:3003`. Se crean registros exclusivos de la ejecución, se guardan sus IDs en variables y se eliminan al finalizar. No reiniciar el servidor mientras corre la colección.

Interpretación de los resultados

Los códigos 400 y 404 de los casos negativos son resultados esperados. Se comprueba que cuerpos incompletos, tipos inválidos, matrículas duplicadas e identificadores inexistentes produzcan errores claros. La consulta posterior a una actualización inválida verifica que el registro no cambie.

La captura del Runner conserva seleccionado un entorno llamado TurnosRed API 2. Las solicitudes de esta colección usan su propia variable `api3BaseUrl` y apuntan al puerto 3003; el nombre del entorno visible no cambia el servidor probado.

Lectura de las evidencias

Las capturas individuales corresponden a comprobaciones efectuadas durante el desarrollo. Hubo reinicios del servidor entre etapas, por lo que no representan una sola secuencia continua de datos. La captura 19 muestra la primera colección de 53 pruebas. La captura 20 muestra la verificación final de 117 pruebas, incluida la regresión de los endpoints heredados.

Se conservan los archivos originales de las 20 capturas. Cada figura indica el método, la ruta y el resultado observado. Las imágenes pueden ampliarse digitalmente para leer el detalle del JSON.

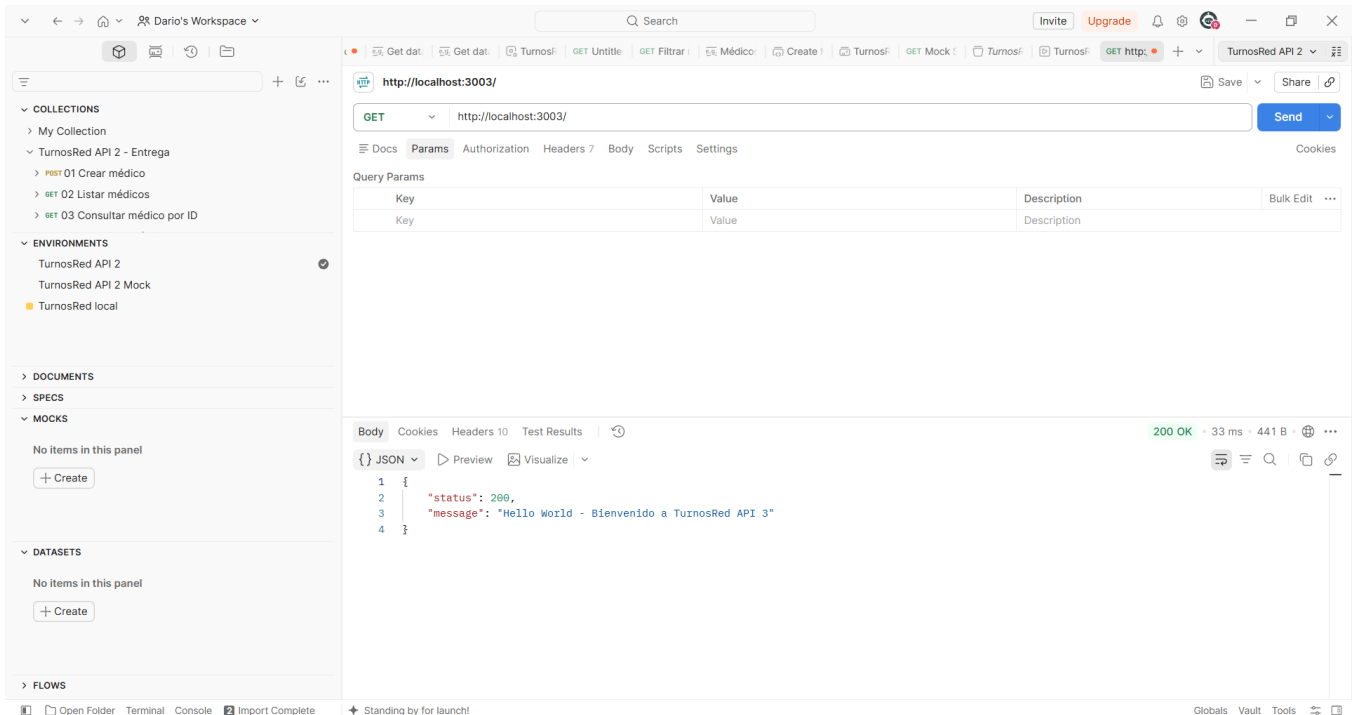
Uso de asistencia de IA

Codex asistió en la implementación de los controladores y servicios, la creación de pruebas y la ejecución de solicitudes en Postman. Darío realizó las primeras 19 capturas durante la revisión paso a paso. Codex guardó la captura 20 de la verificación final. El resultado del Runner corresponde a una ejecución real sobre el servidor local.

Las rutas heredadas `/turnos` y `/medicos` se conservan para compatibilidad con API 2. Sus controladores también se uniformaron con métodos `async`, variable `status`, `try-catch` y `return` explícito. La colección final incluye 21 solicitudes de regresión de estas rutas.

1 Bienvenida

GET / | 200 OK

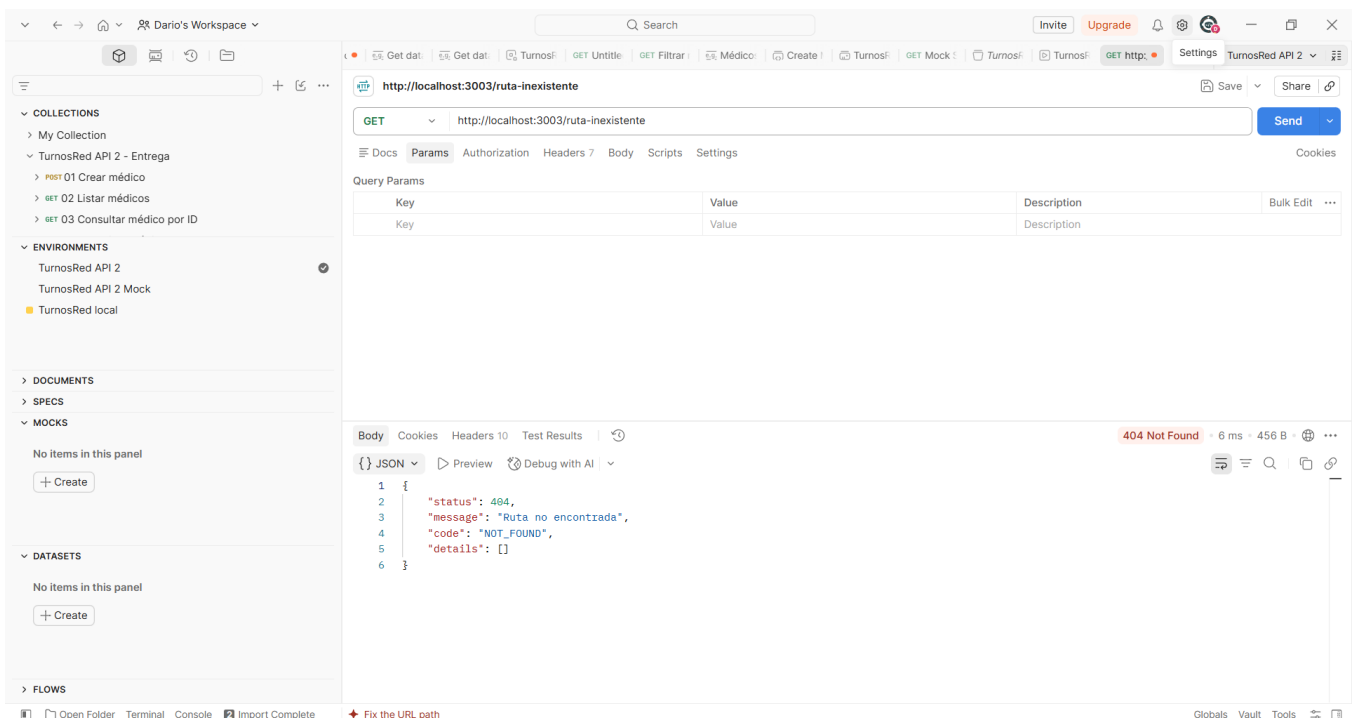


El controlador general devuelve el saludo de la API.

Archivo original: Captura de pantalla 2026-09-21 152120.png

2 Ruta no encontrada

GET /ruta-inexistente | 404 Not Found

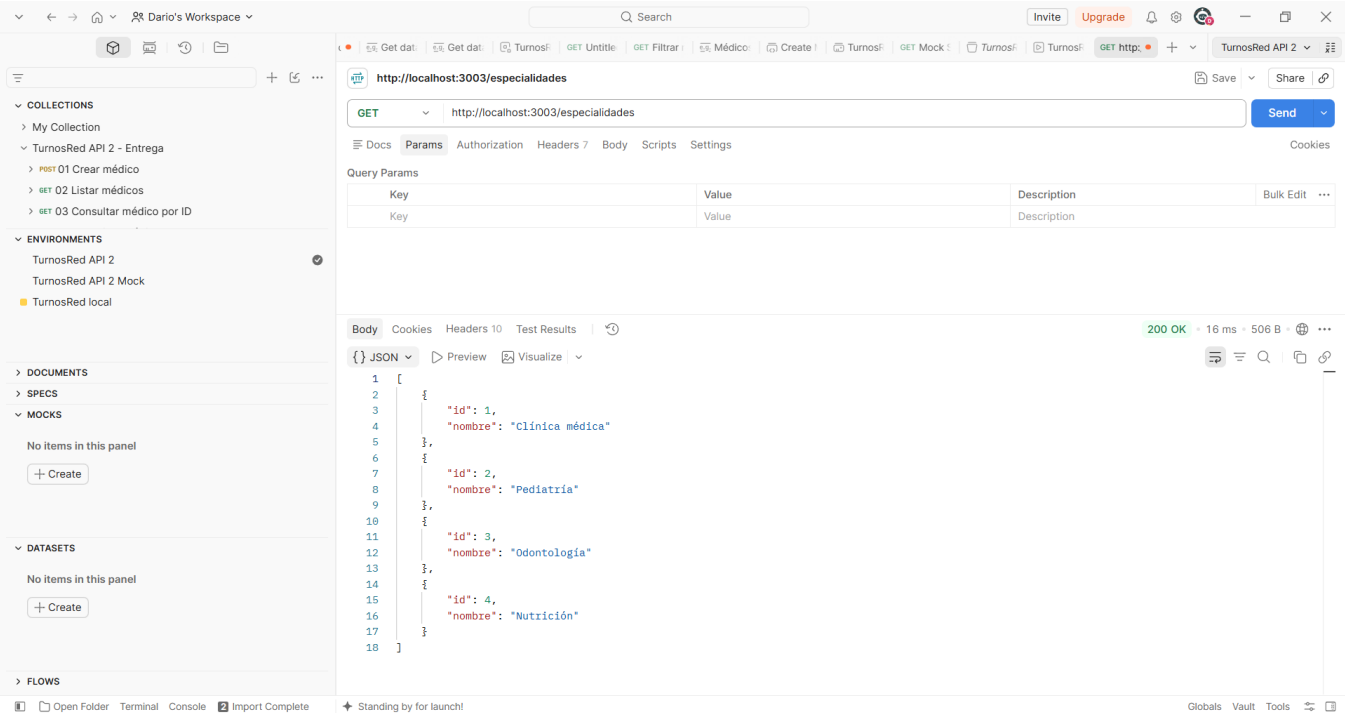


El controlador general informa Ruta no encontrada con code NOT_FOUND.

Archivo original: Captura de pantalla 2026-09-21 152335.png

3 Listado de especialidades

GET /especialidades | 200 OK

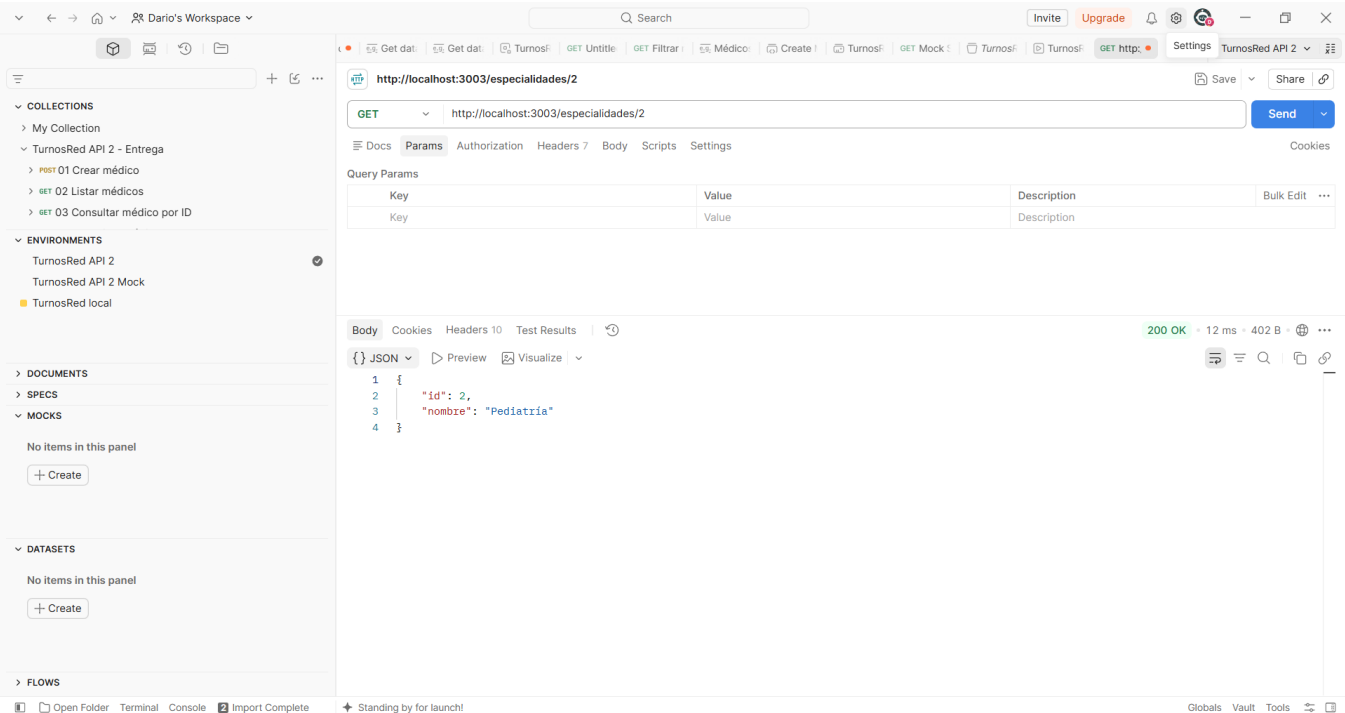


Se devuelve el arreglo de cuatro especialidades iniciales.

Archivo original: Captura de pantalla 2026-09-21 153135.png

4 Consulta de una especialidad

GET /especialidades/2 | 200 OK

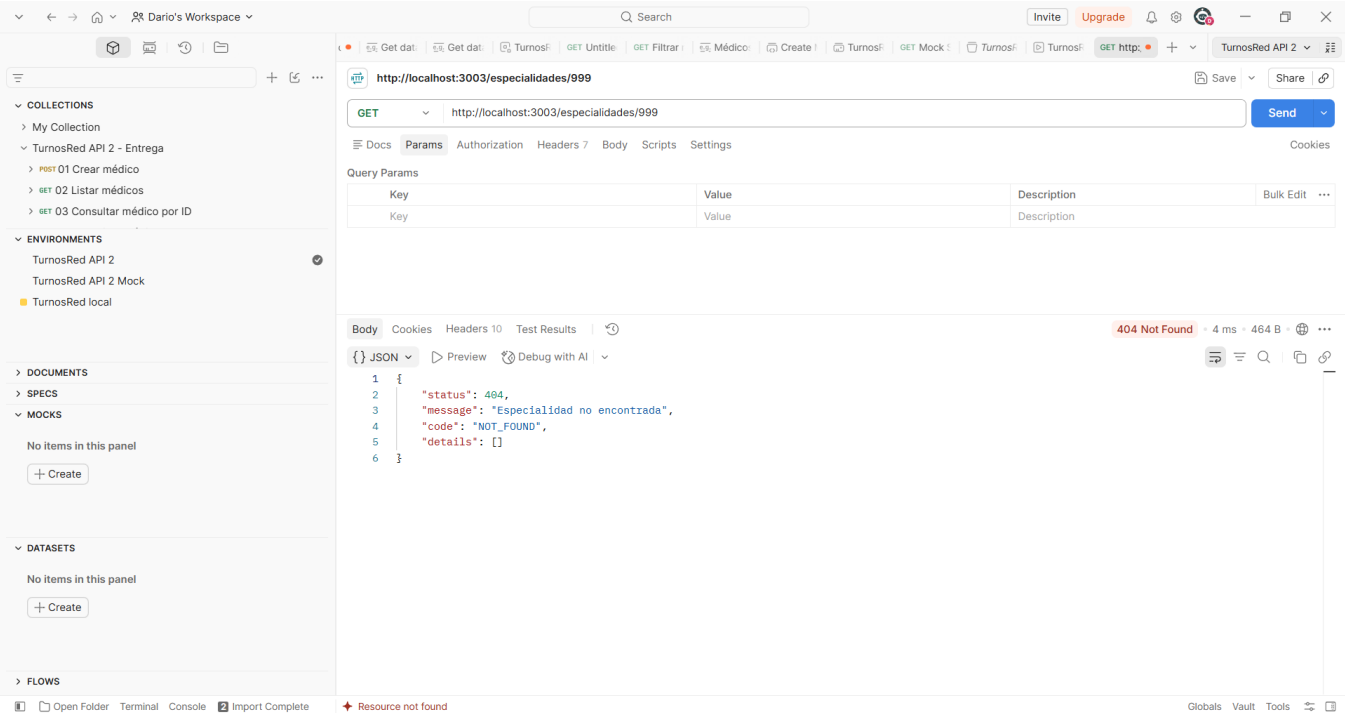


Se recupera Pediatria, identificada con el ID 2.

Archivo original: Captura de pantalla 2026-09-21 153450.png

5 Especialidad inexistente

GET /especialidades/999 | 404 Not Found

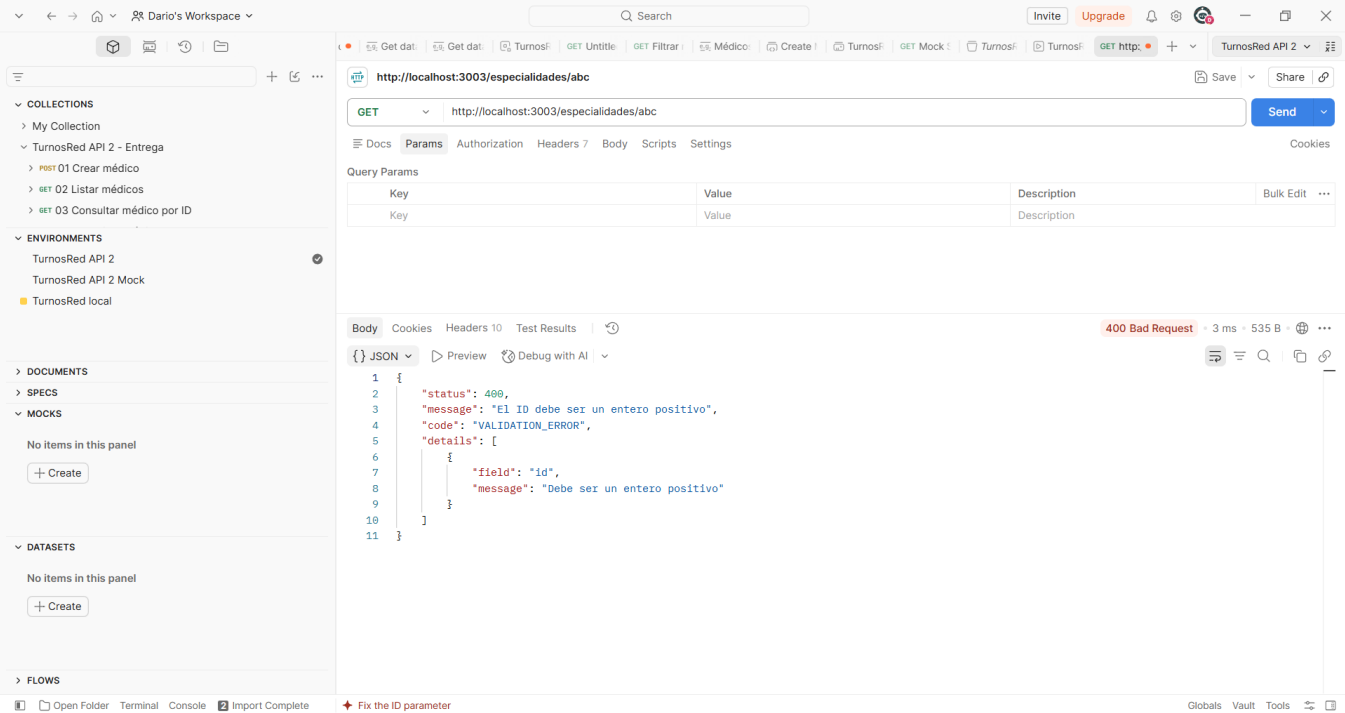


La API informa Especialidad no encontrada.

Archivo original: Captura de pantalla 2026-09-21 153640.png

6 Identificador inválido

GET /especialidades/abc | 400 Bad Request

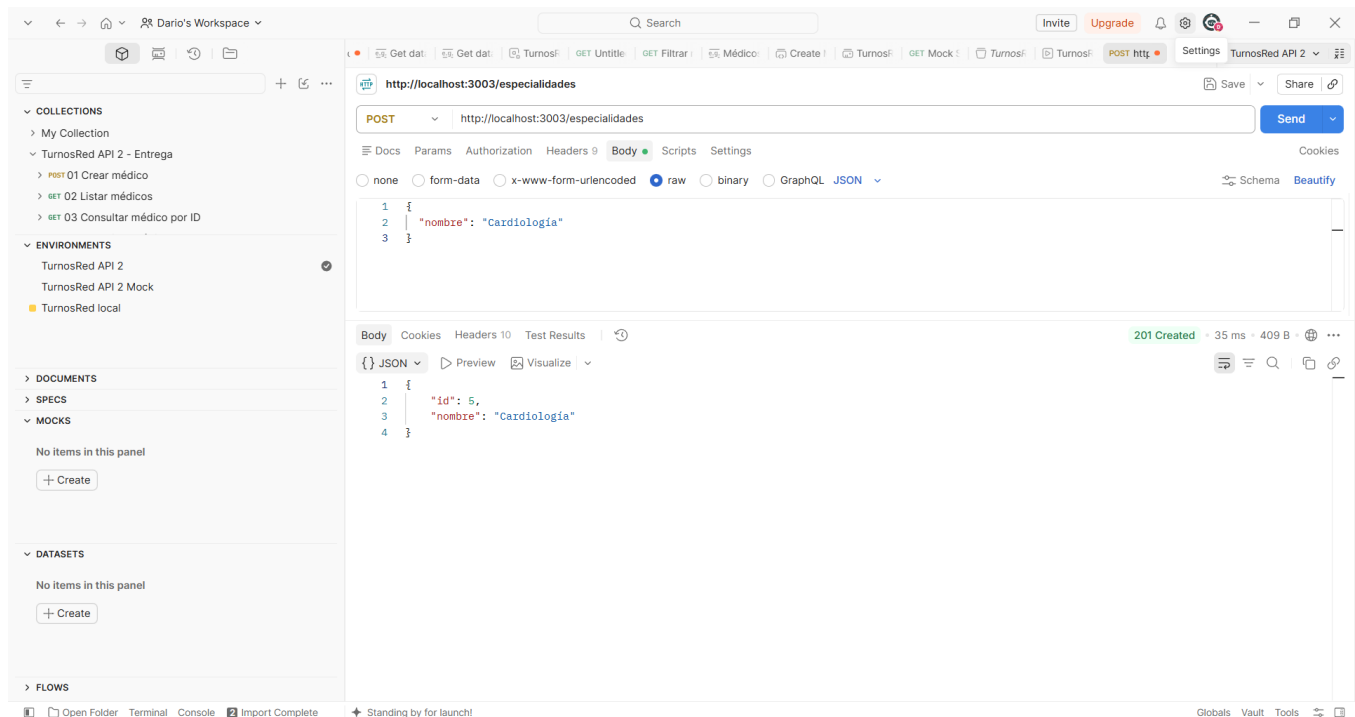


La validación exige un entero positivo y devuelve VALIDATION_ERROR.

Archivo original: Captura de pantalla 2026-09-21 153819.png

7 Creación de especialidad

POST /especialidades | 201 Created

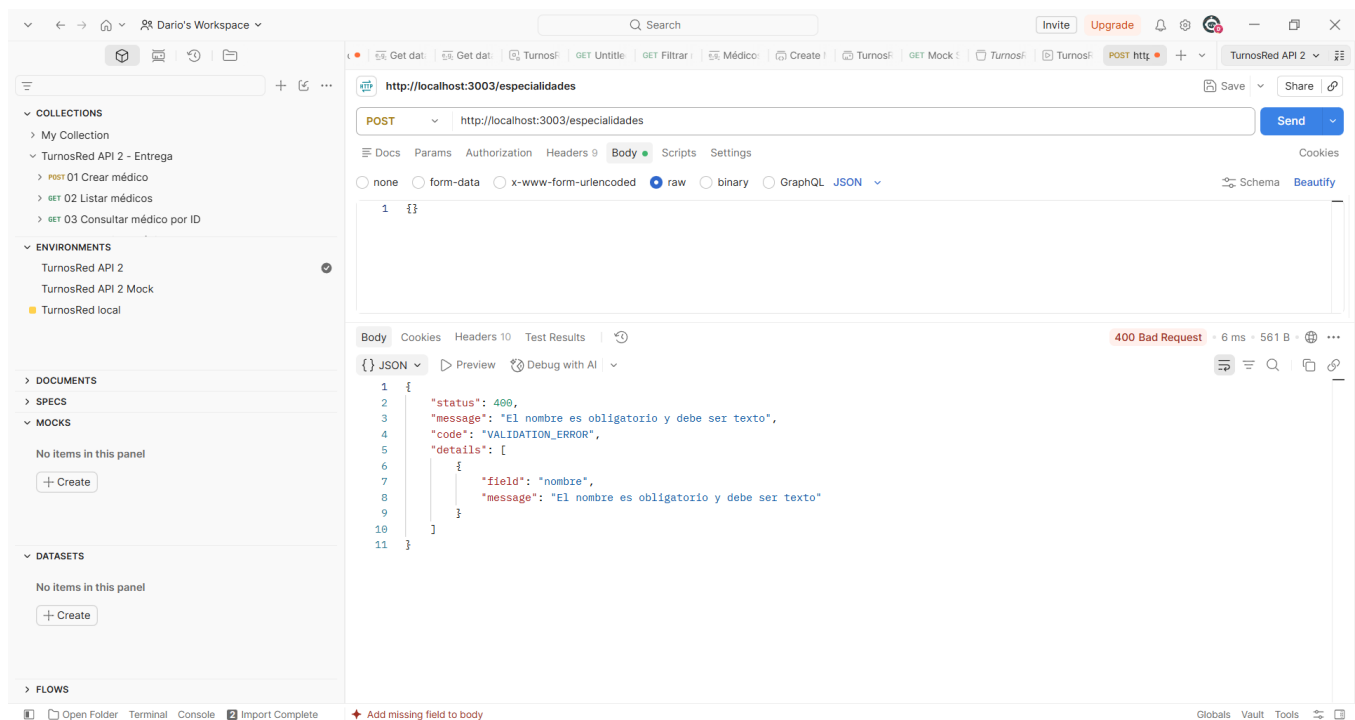


El cuerpo contiene nombre Cardiología. La respuesta asigna el ID 5.

Archivo original: Captura de pantalla 2026-09-21 154041.png

8 Cuerpo incompleto

POST /especialidades | 400 Bad Request

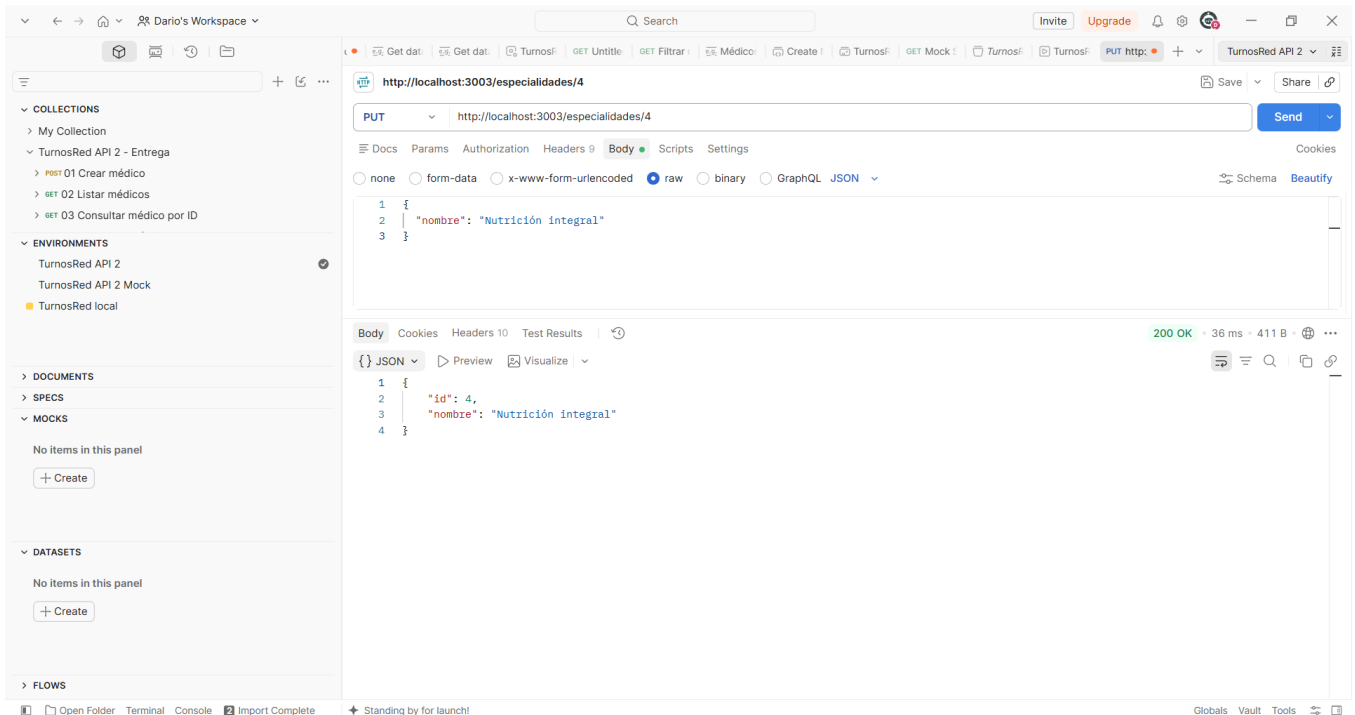


El cuerpo vacío {} se rechaza porque nombre es obligatorio y debe ser texto.

Archivo original: Captura de pantalla 2026-09-21 154241.png

9 Actualización de especialidad

PUT /especialidades/4 | 200 OK

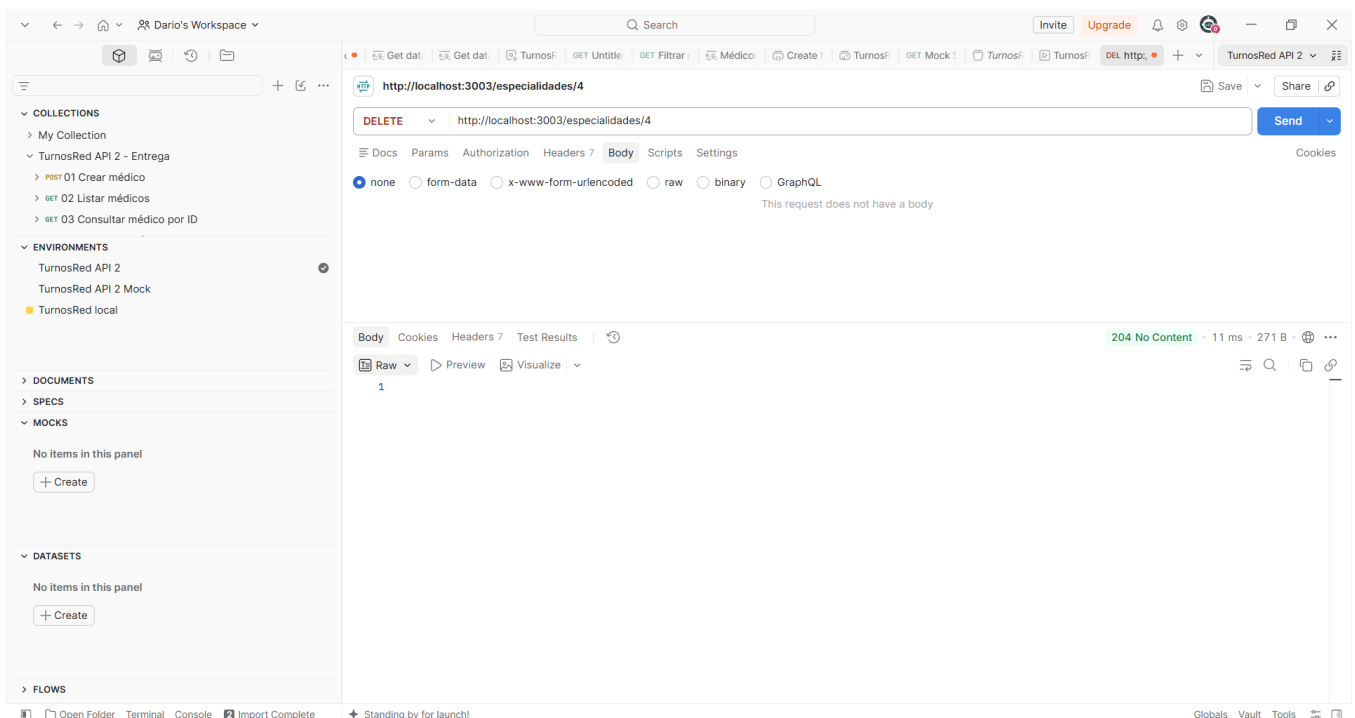


Se devuelve el nombre Nutrición integral y se conserva el ID 4.

Archivo original: Captura de pantalla 2026-09-21 154513.png

10 Eliminación de especialidad

DELETE /especialidades/4 | 204 No Content

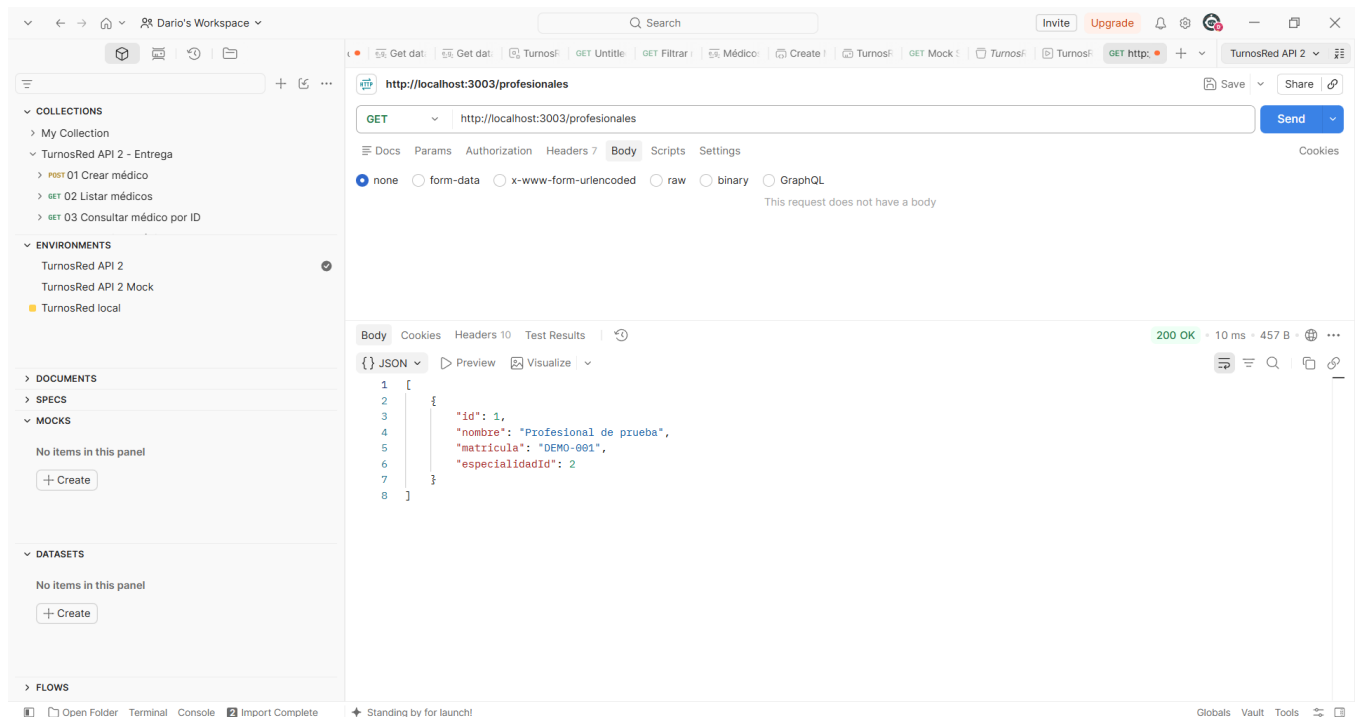


La eliminación devuelve una respuesta sin cuerpo, conforme al estado 204.

Archivo original: Captura de pantalla 2026-09-21 154710.png

11 Listado de profesionales

GET /profesionales | 200 OK

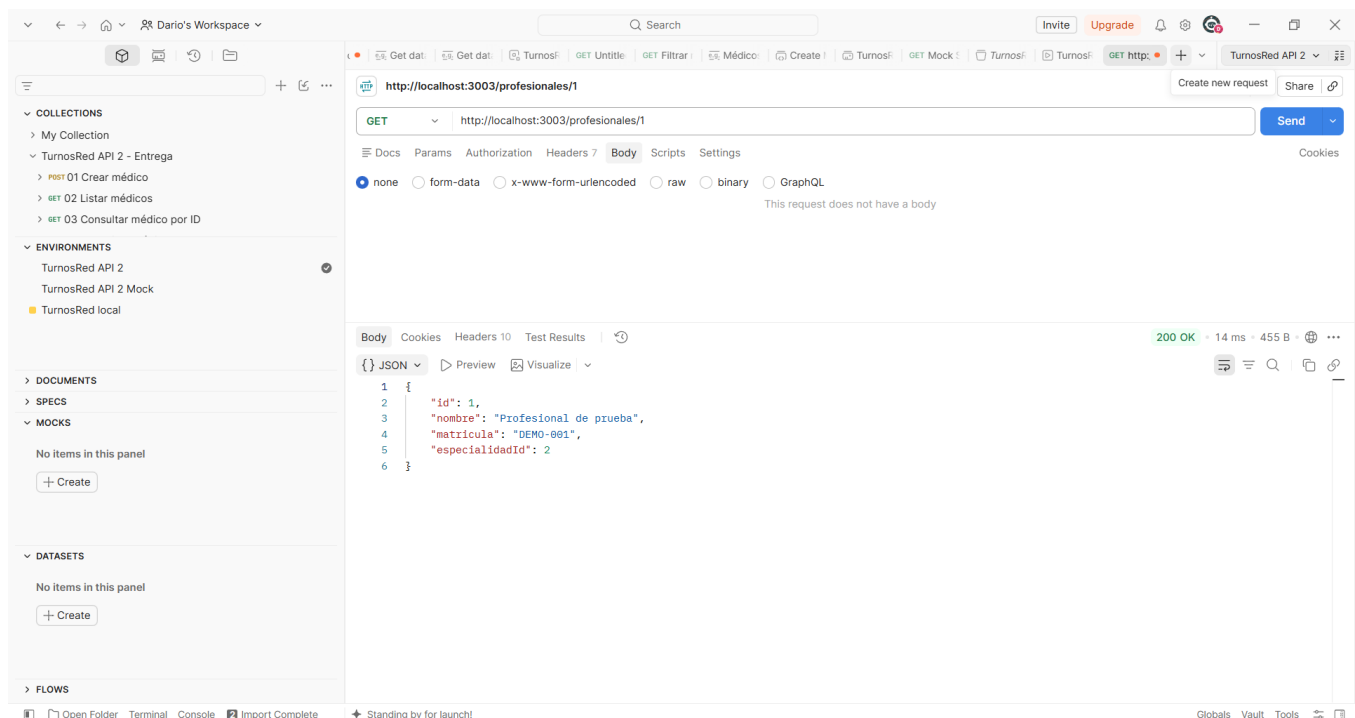


Se obtiene el profesional inicial con matrícula DEMO-001 y especialidadId 2.

Archivo original: Captura de pantalla 2026-09-21 154955.png

12 Consulta de un profesional

GET /profesionales/1 | 200 OK

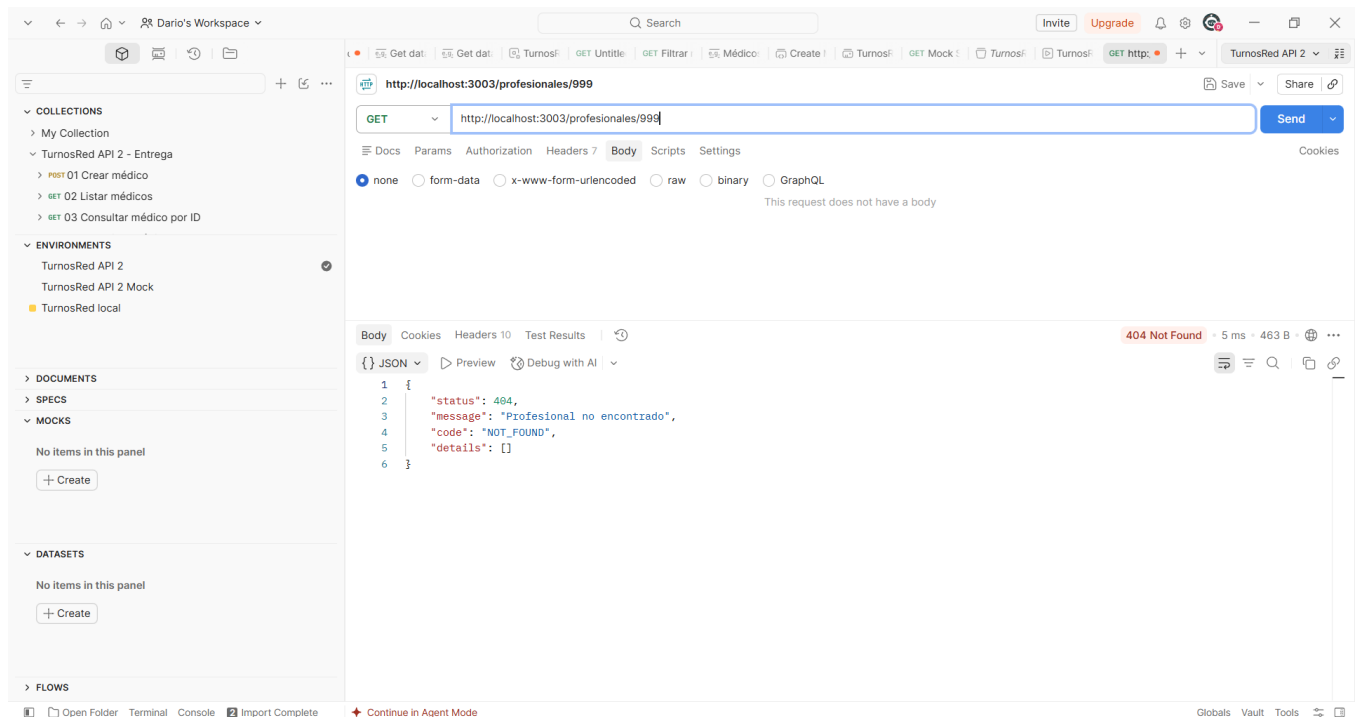


La API devuelve el registro correspondiente al ID solicitado.

Archivo original: Captura de pantalla 2026-09-21 155356.png

13 Profesional inexistente

GET /profesionales/999 | 404 Not Found

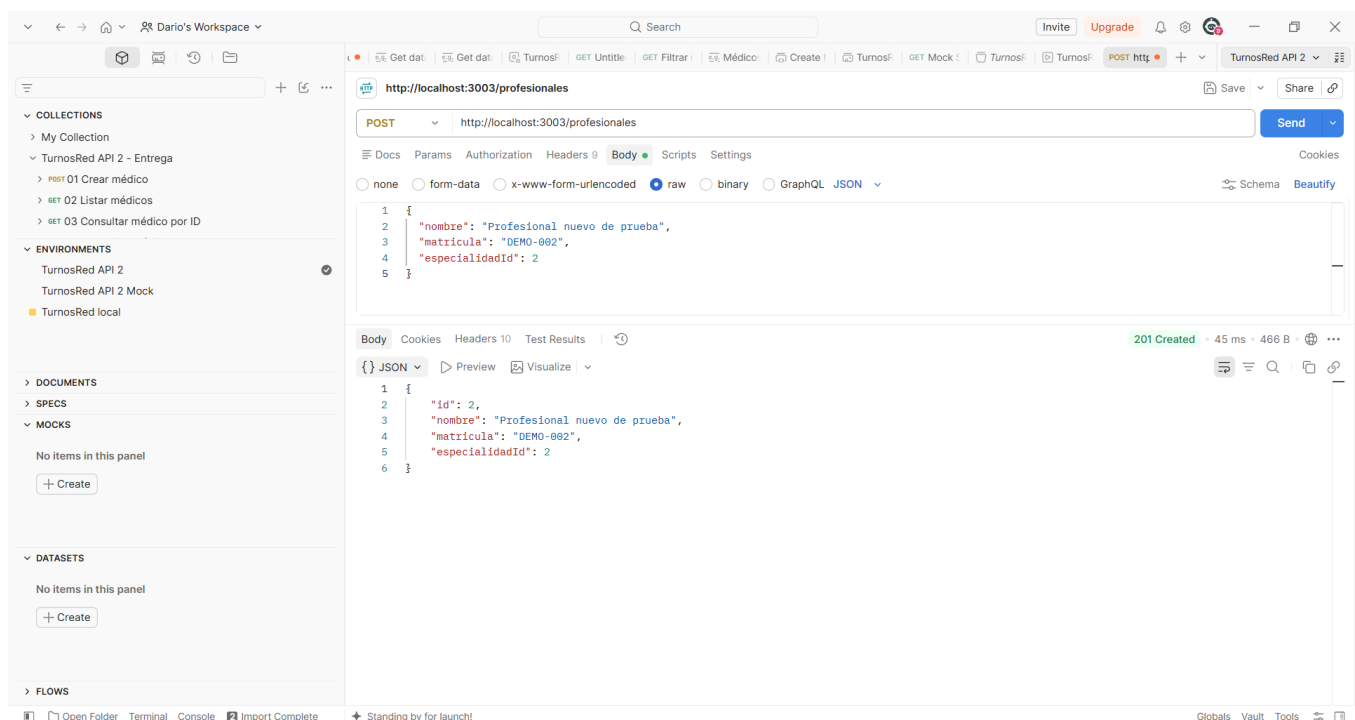


La API informa Profesional no encontrado.

Archivo original: Captura de pantalla 2026-09-21 155620.png

14 Creación de profesional

POST /profesionales | 201 Created

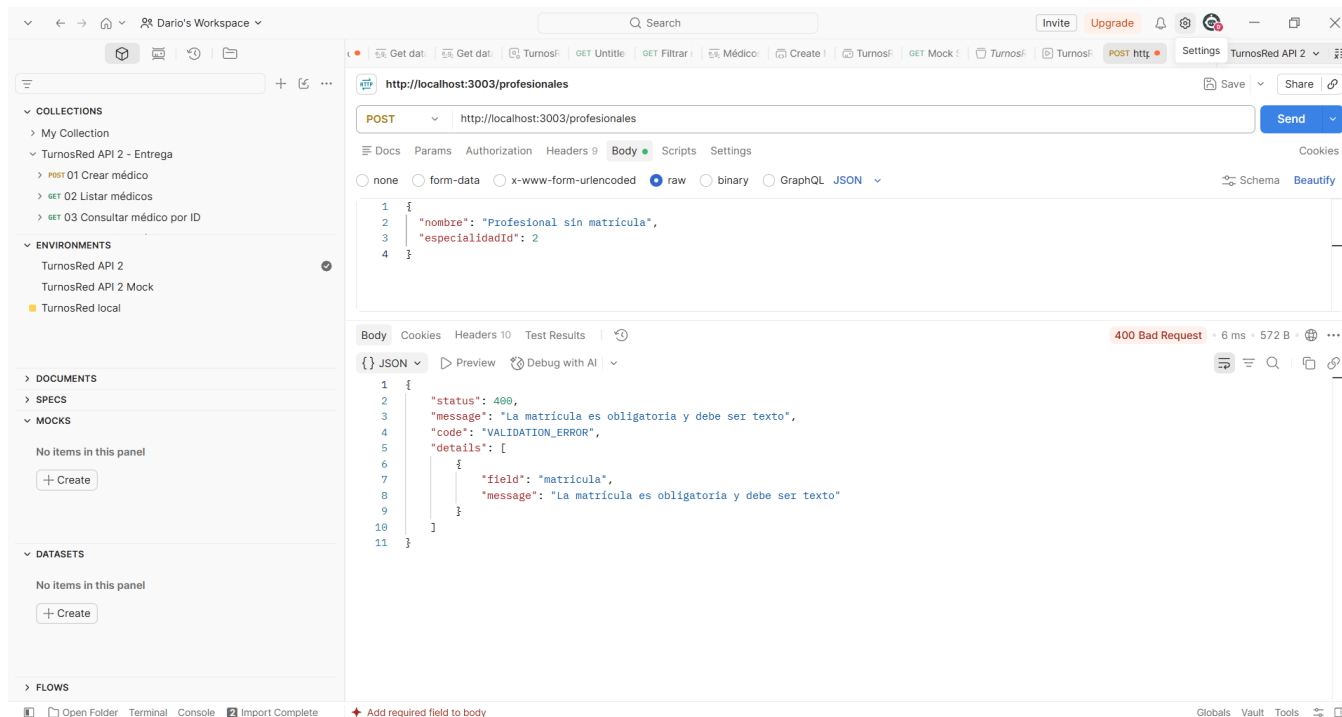


Se registra Profesional nuevo de prueba, con matrícula DEMO-002 y especialidadId 2. Se asigna el ID 2.

Archivo original: Captura de pantalla 2026-09-21 160255.png

15 Matrícula obligatoria

POST /profesionales | 400 Bad Request

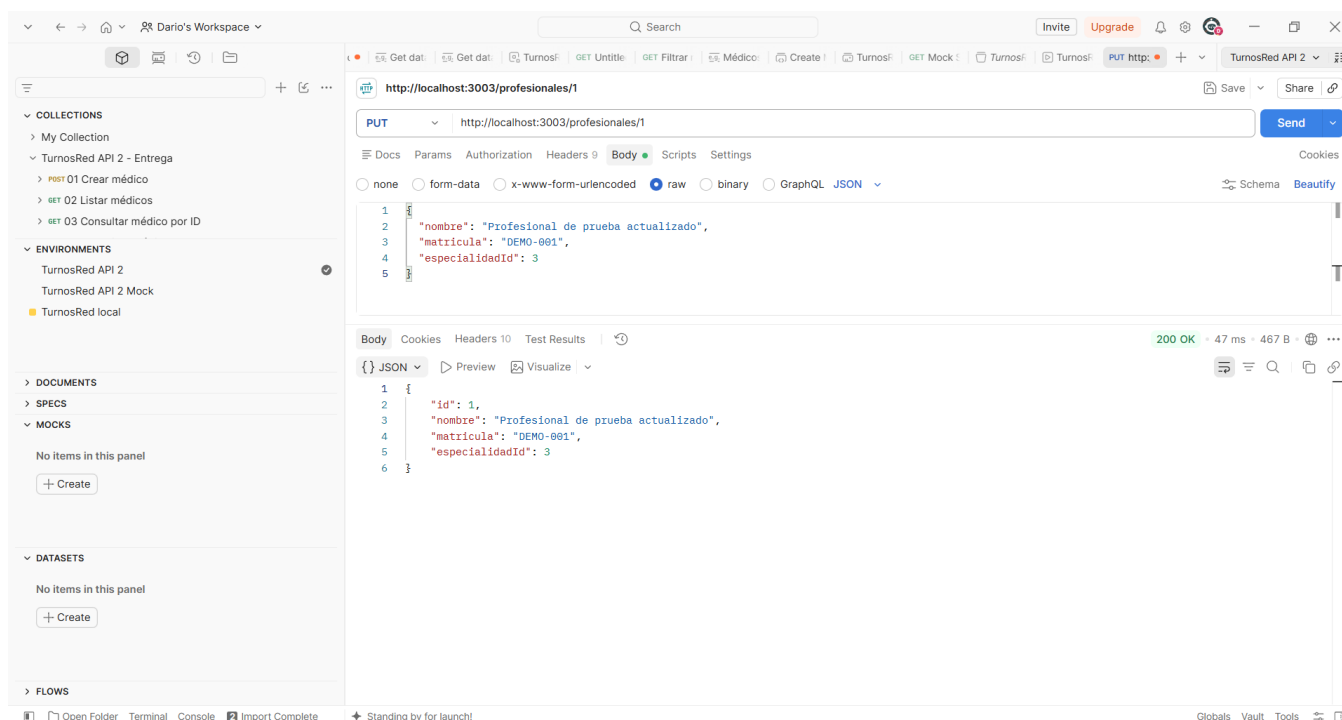


El cuerpo omite matrícula. El error identifica ese campo y explica que debe ser texto.

Archivo original: Captura de pantalla 2026-09-21 160642.png

16 Actualización de profesional

PUT /profesionales/1 | 200 OK

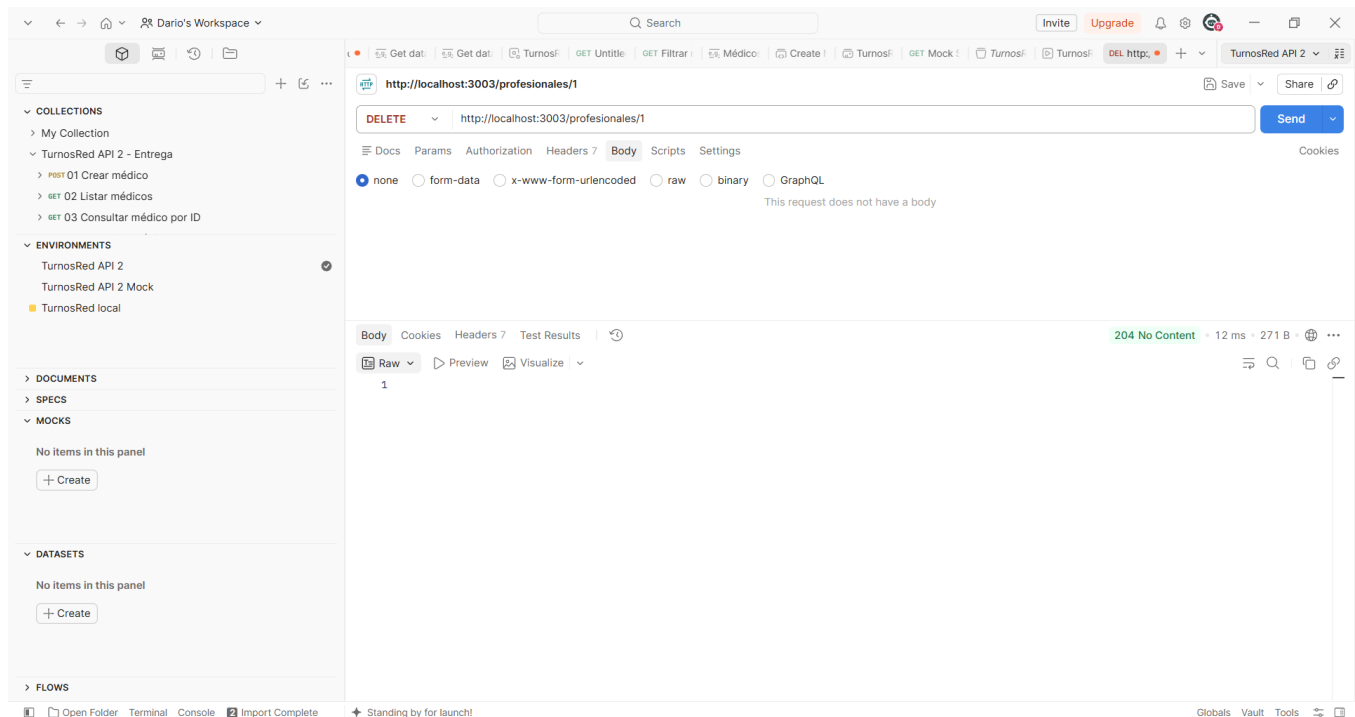


Se modifica el nombre a Profesional de prueba actualizado y la especialidadId a 3; se conserva el ID 1.

Archivo original: Captura de pantalla 2026-09-21 161111.png

17 Eliminación de profesional

DELETE /profesionales/1 | 204 No Content

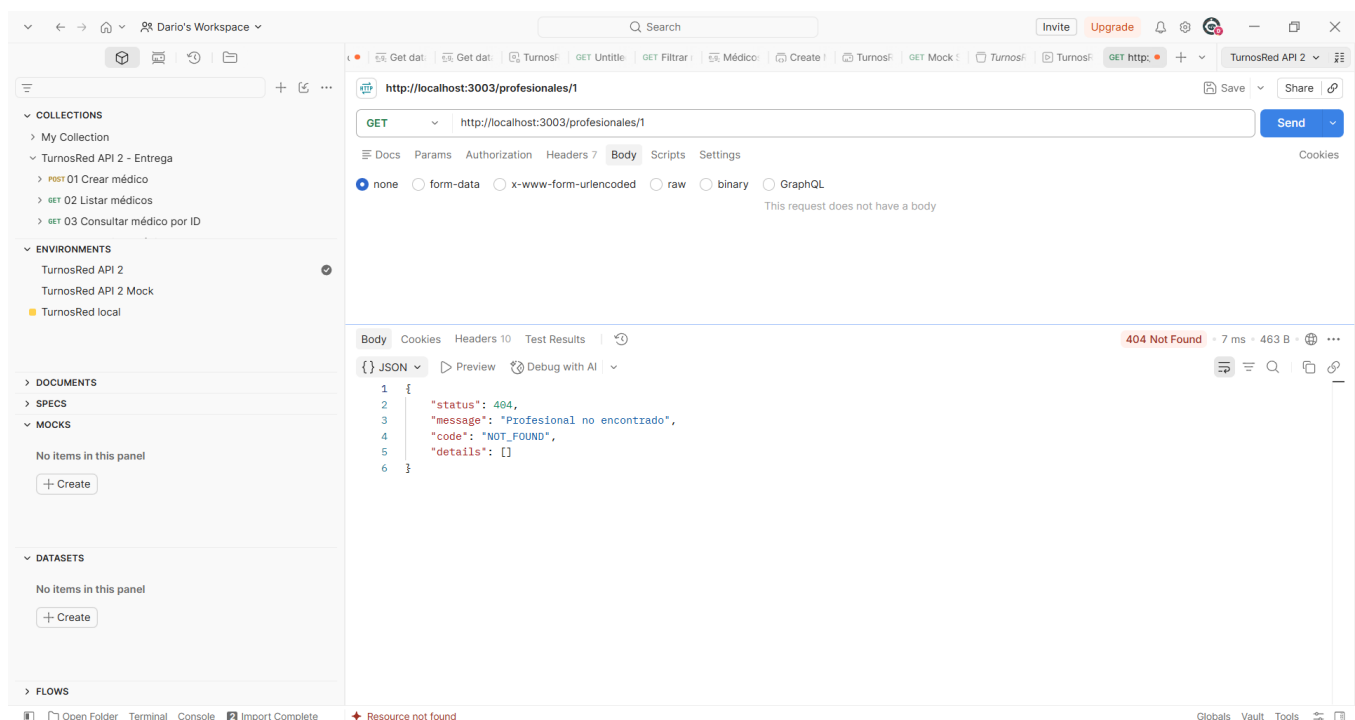


La baja del registro de prueba devuelve un cuerpo vacío.

Archivo original: Captura de pantalla 2026-09-21 161429.png

18 Consulta después de eliminar

GET /profesionales/1 | 404 Not Found

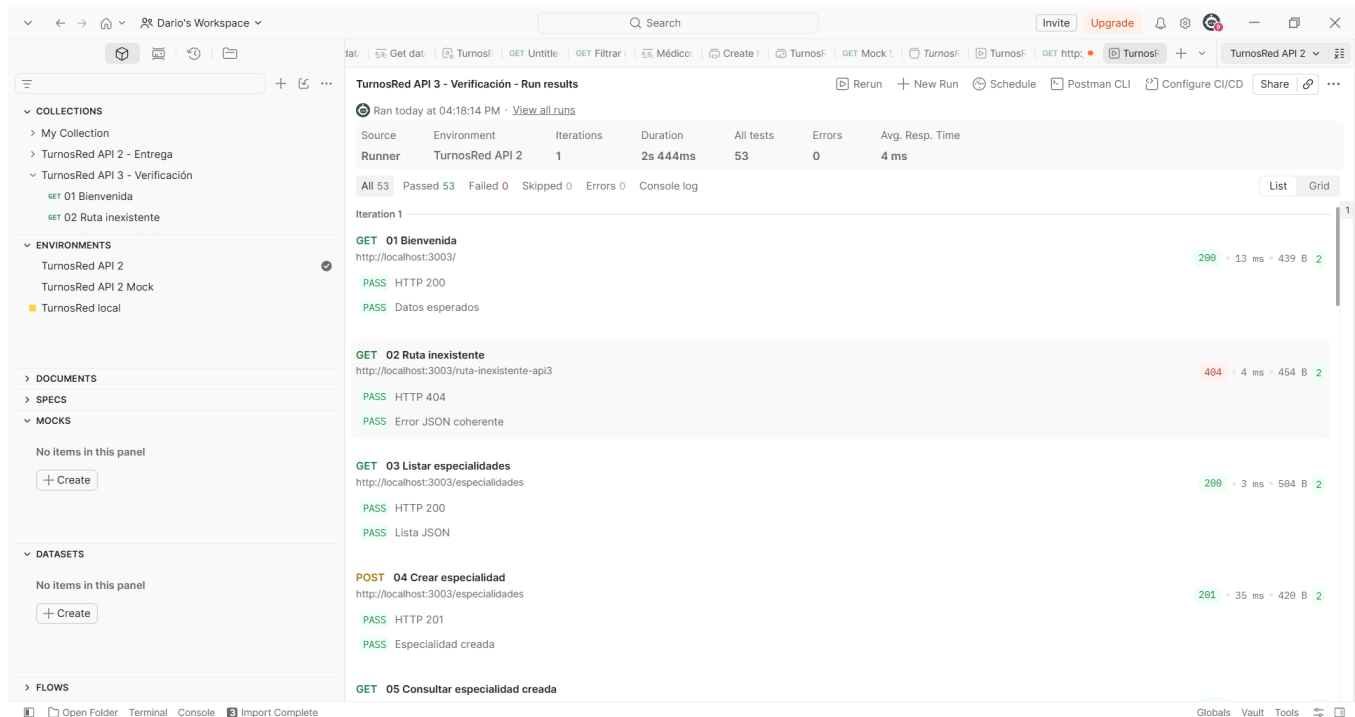


La consulta posterior confirma que el profesional ya no se encuentra en memoria.

Archivo original: Captura de pantalla 2026-09-21 161543.png

19 Ejecución de la colección

Runner de Postman | 53 aprobadas y 0 fallidas

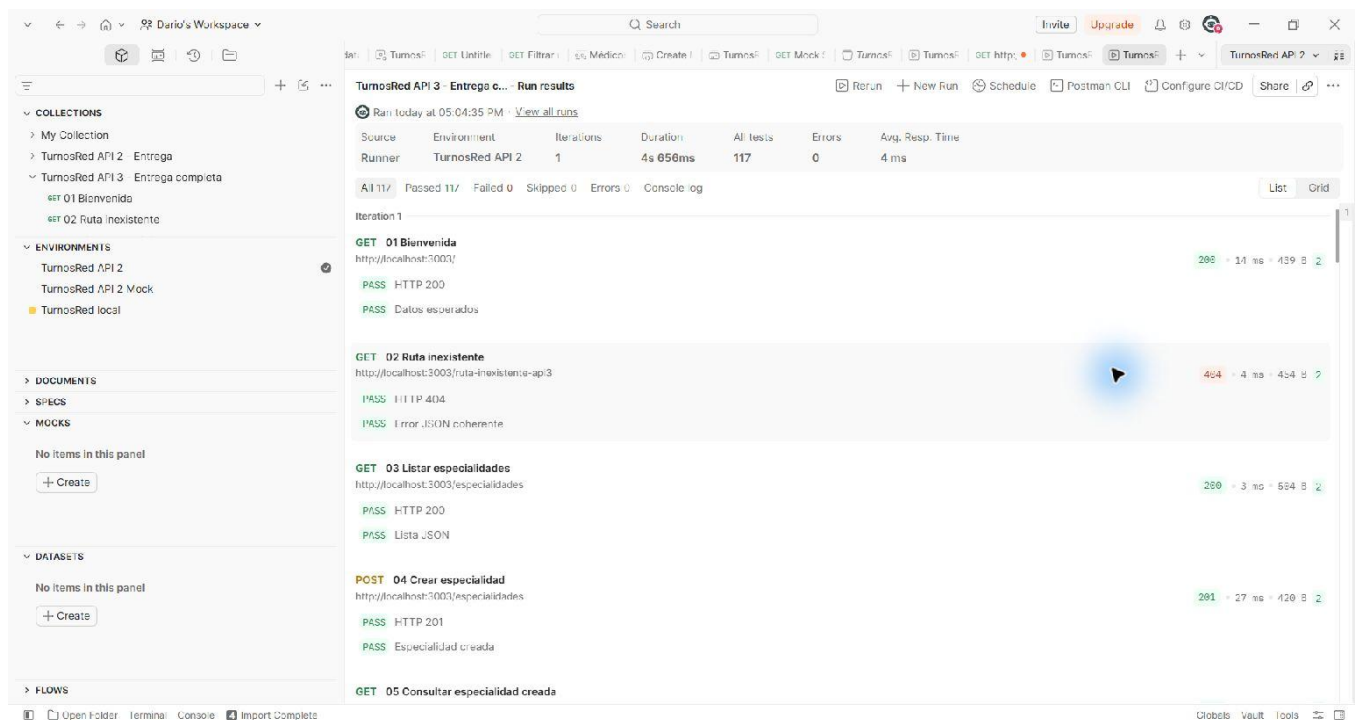


Las 26 solicitudes completan 53 pruebas, sin errores ni pruebas omitidas. Se verifican recorridos exitosos y errores esperados.

Archivo original: Captura de pantalla 2026-09-21 162103.png

20 Verificación final con regresión

Runner de Postman sobre la versión compilada | 117 aprobadas y 0 fallidas



La colección final ejecuta 47 solicitudes: las 26 de API 3 más 21 de regresión de turnos y médicos. Las 117 comprobaciones pasan sin errores ni omisiones.

Archivo original: Captura de pantalla 2026-09-21 170435.png